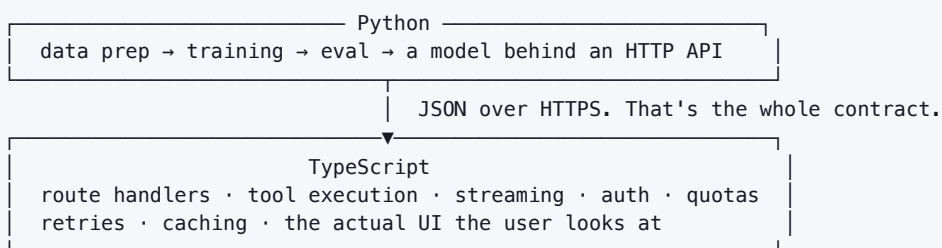


Core Concepts — TypeScript for AI Applications

Track 16.2 — [TypeScript for AI Applications](#) · [16-ai-tech-stack/tracks/typescript/notes/01-core-concepts.md](#)

1. The division of labour

Python trains and serves the model. TypeScript ships the product. This is not a tribal preference; it is where the artifacts live.



The moment the model is a network call, "what language is the model in" stops mattering and "what language is the product in" starts mattering. For anything with a browser in front of it, the answer is TypeScript — because the alternative is two languages, two type systems, two sets of DTOs, and a hand-maintained mapping between them that goes stale the week after you write it.

Where this sits: you already know how models behave ([../04-llm/ \[04-llm\]](#)), how agents loop ([../06-ai-agents/ \[06-ai-agents\]](#)) and what retrieval does ([../08-rag/ \[08-rag\]](#)). This track is the layer that turns those into something a user can click.

2. Why TypeScript actually won this layer

Five concrete reasons, in rough order of how much they matter:

Reason	What it buys you
One language, client to server	A <code>Message</code> type defined once is the same type in the React component, the API route, and the tool implementation. No OpenAPI codegen step, no drift.
Streaming is native	<code>ReadableStream</code> , async iterators and <code>Response</code> are web platform primitives. Python's streaming story requires ASGI plumbing you do not want to own.
The runtimes are where the users are	Cloudflare Workers, Vercel Edge, Deno Deploy, Bun. All run JS at the edge with sub-50ms cold starts. Python at the edge is still a compromise.
Type-safe tool schemas	A Zod schema is simultaneously a validator, a static type, and the JSON Schema you send the model. Three artifacts, one declaration.
The SDKs are first-class	<code>@anthropic-ai/sdk</code> and the Vercel AI SDK are TypeScript-first, not ports. Their types encode the actual API shape, including the discriminated unions.

The honest counter-argument: **do not** rewrite your eval harness, your data pipeline, or your fine-tuning code in TypeScript. Those belong in Python and the ecosystem gap there is not closing. See `../python/` [16-ai-tech-stack/tracks/python].

3. Structural typing: the thing that trips up Java people

TypeScript compares types by *shape*, not by *name*.

```
type Message = { role: "user" | "assistant"; content: string };
function send(m: Message) {}

const anon = { role: "user" as const, content: "hi", traceId: "abc" };
send(anon); // ✅ extra property, fine
send({ role: "user", content: "hi", traceId: "abc" }); // ❌ excess property check
```

The second line fails and the first succeeds, which looks arbitrary until you know the rule: TypeScript runs an **excess property check** only on *object literals assigned directly* to a typed position. It is a typo-catcher, not a type rule. Assign to a variable first and the check disappears.

Consequence for AI code: you can pass an SDK's `MessageParam` anywhere your own `Message` type fits, and vice versa, with no adapters. That is why you should **stop defining your own interface `ChatMessage`** and import the SDK's types instead. A hand-rolled `{ role: string; content: unknown }` is strictly worse than `Anthropic.MessageParam` — it loses the literal union on `role` and the content-block discrimination.

4. Discriminated unions are the load-bearing feature

Almost everything in an LLM API is a tagged union. Model it that way and the compiler does your error handling for you.

```
type ContentBlock =
  | { type: "text"; text: string }
  | { type: "thinking"; thinking: string }
  | { type: "tool_use"; id: string; name: string; input: unknown }
  | { type: "tool_result"; tool_use_id: string; content: string; is_error?: boolean };
```

The shared literal field (`type`) is the **discriminant**. Narrowing on it gives you the right member for free:

```
for (const block of message.content) {
  if (block.type === "tool_use") {
    block.name;    // string ✅
    block.text;    // ❌ Property 'text' does not exist on type '{ type: "tool_use"; ... }'
  }
}
```

Three places this pays off immediately:

1. **Message roles** — `"user" | "assistant" | "system"` as a literal union, not `string`. A typo becomes a compile error instead of a 400 from the API.
2. **Streaming events** — `message_start | content_block_start | content_block_delta | content_block_stop | message_delta | message_stop | error`. A `switch` over these is the core of every stream renderer.
3. **Tool results** — success and failure as separate members, so you physically cannot read `.value` off a failure.

Exhaustiveness checking

This is the single highest-leverage five lines in an AI codebase:

```
function assertNever(x: never): never {
  throw new Error(`Unhandled variant: ${JSON.stringify(x)}`);
}

switch (event.type) {
  case "content_block_delta": /* ... */ break;
  case "message_stop":       /* ... */ break;
  default: assertNever(event); // ❌ compile error listing every case you forgot
}
```

Why it matters *here* specifically: providers add stream event types. When `@anthropic-ai/sdk` ships a new event member, `assertNever` fails your build with the exact name of the event you are silently dropping. Without it, a `default: break` means your UI quietly stops rendering a feature and nobody notices for a month. Turn on `noFallthroughCasesInSwitch` while you are at it.

5. `unknown` vs `any`: the rule for every boundary

`any` disables the type checker. `unknown` demands a check before use.

```
const a: any = JSON.parse(body);
a.messages[0].content.text.toUpperCase(); // compiles. Explodes at runtime.

const u: unknown = JSON.parse(body);
u.messages; // ❌ 'u' is of type 'unknown'
```

Rule: every value that crosses a trust boundary is `unknown` until validated. The boundaries in an AI app are:

- the model's response body (yes, including tool-call `input`)
- the incoming HTTP request
- anything from a database, a queue, or an MCP server (`../../09-mcp/ [09-mcp]`)
- `JSON.parse` of literally anything

Ban `any` with `@typescript-eslint/no-explicit-any` (or biome's `noExplicitAny`). The escape hatch you actually want in the rare hard case is a `// eslint-disable-next-line` with a comment explaining why, not a project-wide allowance.

6. The type-system features you will reach for

Generics — for the tool registry

Without generics, a tool registry collapses every handler's argument to `unknown`. With them, the argument type follows the schema:

```
function defineTool<N extends string, S extends z.ZodType>(def: {
  name: N; input: S; run: (args: z.infer<S>) => Promise<string>;
}) { return def; }
```

See `defineTool` in `../code/typed-tool-calling.ts`. Written inline without the generic, `args` is `unknown` and you are back to casting.

`satisfies` — validate without widening

`as` lies. A type annotation widens. `satisfies` does neither:

```
const MODELS = {
  fast: { id: "claude-haiku-4-5", maxOut: 8192 },
  smart: { id: "claude-opus-5", maxOut: 64000 },
} satisfies Record<string, { id: string; maxOut: number }>;

MODELS.smart.id; // type is "claude-opus-5", not string ✅
MODELS.typo; // ❌ compile error
```

With `: Record<string, ...>` you would get `string` back and lose the keys. With `as` you would get no checking at all. `satisfies` gives you both. Use it for every config object, model table, and tool map you write.

Template literal types — for typed IDs and event names

```
type ToolEvent = `tool:${"start" | "result" | "error"}`; // "tool:start" | ...
type ProviderModel = `${"anthropic" | "openai"}/${string}`;
```

Useful, but keep it boring. Template literal types compile slowly and read badly past one level of nesting. If you find yourself writing a type-level parser, write a runtime parser instead.

7. Runtime validation, or: why your types are worthless here

This is the central point of the whole track, so it gets its own heading.

A TypeScript type does not exist at runtime. The model has never seen your tsconfig. `tsc` cannot check a network response.

Everything you know about a model's output is a *hope*. Even with `strict: true` on a tool definition, even with structured outputs, you are one provider incident away from a truncated JSON body. The failure mode is not theoretical: `../code/typed-tool-calling.ts` simulates a "well-behaved" model turn and **3 of 5** tool calls are malformed — a bad enum value, a missing required field, a stringified number, and an invented tool name.

Zod is the answer, and it is not close

```
import { z } from "zod";

const TicketInput = z.object({
  title: z.string().min(3),
  severity: z.enum(["low", "medium", "high"]),
  assignee: z.string().nullable(),
});

type TicketInput = z.infer<typeof TicketInput>; // the static type, derived
const parsed = TicketInput.safeParse(modelOutput); // the runtime check
const schema = z.toJSONSchema(TicketInput, { io: "input" }); // what you send the model
```

One declaration, three artifacts. That is the whole pitch. The alternative — a TS interface, a hand-written validator, and a hand-written JSON Schema — has three places to forget to update and no compiler linking them.

Practical rules:

- **safeParse, not parse.** `parse` throws; in an agent loop a throw is a dead turn. `safeParse` returns a tagged union you feed back to the model.
- **.describe() every field.** It becomes the `description` in JSON Schema, which is the single cheapest way to improve tool-call accuracy.
- **Prefer .nullable() over .optional()** for model-facing fields. Models are much better at emitting an explicit `null` than at omitting a key.

- **io: "input" when generating tool schemas.** A field with `.default()` has a different input type (optional) than output type (always present). Get this backwards and you tell the model a field is required when it is not.
- **`z.prettifyError(err)`** turns issues into a human-readable string. Send that string straight back as the `tool_result` content and the model usually fixes itself on the next turn.

zod-to-json-schema vs z.toJSONSchema

Historically you needed the `zod-to-json-schema` package. **Zod v4 ships `z.toJSONSchema()` natively — use it.** The trap: point `zod-to-json-schema` at a Zod v4 schema and it does not throw. It reads v3-era internals, finds nothing, and returns `{}`. An empty schema means the model receives *zero* constraints and invents field names. The demo is in §2 of `../code/typed-tool-calling.ts` — run it and look at the empty `definitions` object.

Keep `zod-to-json-schema` only if you are pinned to Zod v3.

8. Errors as values: Result types

Exceptions are the wrong shape for an agent loop, because a thrown tool error kills the turn instead of informing the model.

```
type Result<T, E> = { ok: true; value: T } | { ok: false; error: E };
```

It is a discriminated union, so `if (!r.ok)` narrows `r.error` and the compiler refuses to let you read `r.value` on the failure branch. Pair it with a closed union of error kinds:

```
type ToolError =  
  | { kind: "unknown_tool"; name: string }  
  | { kind: "invalid_arguments"; name: string; detail: string }  
  | { kind: "tool_threw"; name: string; detail: string };
```

Now `assertNever` forces you to produce a repair message for every failure mode. Libraries exist (`neverthrow`, `effect`) but a 1-line type alias covers 95% of the need. Reach for `effect` only if you are already buying into its scheduler and dependency-injection model — it is a framework, not a utility.

What to still throw: programmer errors (a missing env var at boot, an impossible state). Those should crash loudly. Reserve `Result` for expected failures the caller — or the model — can act on.

9. Where this returns

Idea here	Where it comes back
Discriminated union on <code>type</code>	§2 of 02-streaming-runtimes-and-hygiene.md — the stream event switch
Zod schema → JSON Schema	../06-ai-agents/ [06-ai-agents] — tool definitions for the agent loop
Zod schema → MCP tool	../09-mcp/ [09-mcp] — the MCP TS SDK takes Zod schemas directly
<code>unknown</code> at every boundary	../13-ai-security/ [13-ai-security] — untrusted model output as an injection vector
<code>Result</code> + repair messages	../15-ai-evals/ [15-ai-evals] — tool-call validity as an eval metric
Structured output	../10-ai-architecture/ [10-ai-architecture] — contracts between services
The Python side of the wire	../python/ [16-ai-tech-stack/tracks/python]